EM1 — Experience Deep Dives

What I Built at Samay + Epik | Technical + Behavioral

How to use this doc:

- Tuesday PM: Read Samay technical, then Epik technical. Practice behavioral stories **out loud**.
- Samay = primary depth. Ground every EM1 design answer in a Samay parallel.
- Epik = diversity of context (team leadership, LangGraph in production, eval harness, TypeScript).
- Behavioral stories are consolidated in Part C — organized by question type, not by company.

PART A — SAMAY / TRUSTX: TECHNICAL

A1: WHAT TRUSTX WAS (know this cold)

One-liner: Compliance automation SaaS. I was the sole engineer. Built from 0 → 3 paying B2B customers. Infrastructure, AI pipeline, React UI — all of it.

The problem: Enterprise compliance teams (SOC2, NIST 800-53, ISO 27001) manually pulled evidence from GitHub, Jira, AWS Config, Confluence, Slack — then wrote assessment summaries in Word. TrustX automated collection, ran LLM evaluation, and surfaced results with citations for the human auditor to review and sign off.

Architecture at a glance:

- 15+ AWS CDK stacks: VPC, ECS, Lambda suite, RDS PostgreSQL, Neo4j on EC2, S3, Step Functions, AppRunner, Secrets Manager
- Per-tenant physical isolation: separate S3 bucket, Neo4j DB, RDS schema per customer
- AppRunner Flask API: NL → Cypher query translation via LangChain + OpenAI
- Evidence collection: 6-source Lambda (GitHub, Jira, AWS Config, Confluence, Slack, file uploads)
- LLM scoring: AWS Bedrock (Llama 3.3 70B), Pydantic output schema, evidence-gating
- React dashboard: controls list, evidence management, document editor, compliance report generation
- n8n agentic workflows for compliance report assembly

A2: EVIDENCE-GATING — THE CORE AI PATTERN

Most directly mappable to EM1 Smart References + crosswalk evaluation

The problem: An LLM scoring a compliance control can return PASS with a plausible-sounding justification that has no basis in the actual evidence. In compliance, a hallucinated PASS is worse than no answer.

The constraint:

```
class ControlEvaluation(BaseModel):
    verdict: Literal["PASS", "PARTIAL", "FAIL", "INCONCLUSIVE"]
    evidence_quote: str       # must be non-empty AND verified verbatim
    source_reference: str     # e.g. "Jira ticket INFRA-221, 2024-03-15"
    gaps: list[str]
    confidence: float

# Post-processing gate — runs before any DB write:
if not evidence_quote or evidence_quote not in retrieved_context:
    evaluation.verdict = "INCONCLUSIVE"
    evaluation.evidence_quote = ""
```

INCONCLUSIVE is the honest answer when the system lacks grounded evidence. It routes to the human auditor with a flag. The auditor prefers "I couldn't find it" over a confident wrong answer.

Why this maps to EM1: Smart References = evidence-gating on FEMA guidelines. Evidence quote = verbatim sentence from the plan. Page reference = assembled from chunk provenance metadata, not extracted from LLM output. The LLM cannot fabricate a PASS with a non-existent page 94 quote.

A3: MULTI-TENANT ISOLATION — PHYSICAL VS. LOGICAL

What I built (physical isolation): Every tenant got their own S3 bucket, Neo4j DB instance, RDS schema, and Secrets Manager entry. A code bug cannot cause cross-tenant data exposure — the infrastructure enforces it before any application logic runs.

The tradeoff: Higher infra cost, more complex CDK (one nested stack per onboarding), operational overhead of N DB instances. Right choice at 3–10 customers. Reconsider at 100+.

For EM1 (logical isolation is right):

- `agency_id` FK on every table + PostgreSQL Row-Level Security: `USING (agency_id = current_setting('app.current_agency_id')::UUID)`
- JWT → API middleware injects `agency_id` → every downstream call is parameterized by it
- pgvector: `agency_id = $1` filter on every retrieval query
- Even if app code has a bug and omits a filter, the RLS policy catches it

The comparison I make in the room: "Samay's model was right for that context — early B2B, trust not yet established. EM1's logical isolation is right for this context — same discipline, different tradeoff."

A4: GRAPH DATABASE DECISION — NEO4J VS. POSTGRESQL

Use as technical depth AND for the "hard architectural decision" behavioral question

The problem: TrustX's identity module tracked multi-hop relationships: user → roles → IAM policies → AWS resource actions → S3 buckets. Compliance question: "who can access this bucket, through what permission chain?" Required 4–6 hop traversals.

I built it both ways: PostgreSQL recursive CTE — correct, readable, 800ms on a medium-complexity chain. Neo4j Cypher traversal on the same graph — 20ms. 40x difference.

The decision: Neo4j for the identity graph only. Everything else stays in PostgreSQL. Per-tenant Neo4j instances for isolation.

The cost accepted: Two weeks of slower velocity — learning Cypher, deployment patterns, backup/restore, index tuning. New CDK construct for Neo4j on EC2. Data sync between RDS control plane and Neo4j.

The principle I apply: I don't add architectural complexity because it's interesting. I add it after the simpler version demonstrably fails. The decision log I wrote: *"Adding Neo4j. Justification: PostgreSQL CTE = 800ms, unacceptable for interactive queries. Neo4j = 20ms. Accepted cost: Cypher ramp, CDK construct, per-tenant instances."*

Bridge to EM1 Deconflict: Cross-plan contradiction detection has the same graph shape — sections as nodes, `contradicts` edges. Start with a PostgreSQL adjacency table. Reach for Neo4j only when the traversal queries can't be expressed efficiently in SQL. Same decision rule.

A5: AWS CDK INFRASTRUCTURE DEPTH

"What have you actually built with CDK?"

The 15+ stacks:

- `NetworkStack` — VPC, subnets, security groups, NAT gateway
- `DatabaseStack` — RDS PostgreSQL Multi-AZ, Secrets Manager rotation Lambda
- `Neo4jStack` — EC2, EBS, user data script for Neo4j install + config
- `LambdaStack` — 6 evidence-collection functions, SQS queues, EventBridge nightly schedule, least-privilege IAM per function
- `StepFunctionsStack` — Pipeline orchestration: collection → evaluation → report
- `AppRunnerStack` — Flask API in ECR, AppRunner service, IAM role for RDS VPC access
- `FrontendStack` — S3 + CloudFront + ACM cert

- `SecretsStack` — Per-tenant Secrets Manager entries, rotation Lambda
- `TenantOnboardingStack` (nested, parameterized) — per-tenant S3 bucket, RDS schema init, Neo4j DB init, IAM policies

Bugs I debugged:

- **Circular dependency** between `DatabaseStack` and `LambdaStack` — Lambda needed the DB endpoint for an env var; DB needed the Lambda ARN for a custom resource. Fix: SSM Parameter Store as the decoupling layer. Each stack writes outputs to SSM; consumers read from SSM, not direct cross-stack references.
- **Lambda native dependency bundling** (psycopg2, cryptography) — needed Docker bundling, not the default Python bundler.
- **AppRunner → RDS connectivity** through VPC — AppRunner VPC connector, security group ingress rule on RDS, route table config.

The Terraform translation: Same discipline. Provider/resource/data source/output = CDK construct/resource/lookup/CfnOutput. The hard part — understanding resource dependency graphs, what triggers replacement vs. update, how to handle stateful resources — is shared.

A6: PROFILE-360 PIPELINE — STEP FUNCTIONS ORCHESTRATION

> *"Walk me through a data pipeline you built from scratch."*

What it did: Nightly knowledge graph refresh. Pulled compliance-relevant data from GitHub, Jira, Entra ID, formatted it, and wrote it into each tenant's Neo4j graph DB so the identity graph stayed current.

Architecture:

```
EventBridge (daily 2 AM UTC)
  → Step Functions state machine
      Stage 1 (Parallel): EntraDataFormatting Lambda + JiraDataFormatting Lambda
      Stage 2 (Parallel): EntraToNeo4j + GithubToNeo4j + JiraToNeo4j
      → PipelineSuccess | DataFormattingFailure | Neo4jIngestionFailure
```

Two-stage design was deliberate: formatting and ingestion are separate concerns. A Jira formatting failure shouldn't fail the Entra ingestion that already succeeded. Each stage had its own failure state.

The CDK migration: Originally deployed with shell scripts — Lambdas created manually in the AWS console, state machine defined in the UI. Migrated everything to CDK. The hard part: the Step Functions state machine is JSON with Lambda ARN references, and those ARNs don't exist until CDK synthesizes. Built a `WorkflowLoader` TypeScript class that reads the state machine JSON, replaces ARN placeholders with `{lambda.functionArn}` values from the CDK construct tree. State machine definition and Lambda code are now in sync by construction.

Multi-tenant execution: Each Lambda received tenant context in the Step Functions event payload — resolved the tenant → fetched Neo4j credentials from Secrets Manager → wrote to that tenant's

graph. One pipeline execution per tenant per night.

A7: JWT-TO-NEO4J CREDENTIAL CHAIN — MULTI-TENANT PLUMBING

"How did you implement multi-tenancy end-to-end?"

Every API request went through a 5-step credential resolution:

1. **Auth0 JWT validation** — AppRunner middleware validates signature against Auth0's public key. Extracts `email` from token claims.
2. **Tenant resolution from RDS** — `SELECT tenant_id, schema_name FROM public.tenants WHERE email = $1`. Maps Auth0 user to their tenant. Only step that touches the shared `public` schema.
3. **Secrets Manager lookup** — Key: `trustx/{tenant_id}/neo4j`. Returns `{uri, user, password}` for that tenant's Neo4j instance. Credentials are never shared across tenants.
4. **Neo4j connection** — Connect to tenant-specific graph DB. All Cypher queries run against this connection only.
5. **PostgreSQL schema switching** — `SET search_path = {tenant_schema_name}`. RLS provides a second enforcement layer.

The cold-start problem: Initial implementation loaded all tenant mappings at `create_app()` time — 3–4 second RDS query blocked AppRunner initialization. AppRunner health checks were timing out. Fix: lazy loading with a `_tenant_mappings_loaded: bool` flag. First request triggers the RDS load; subsequent requests hit in-memory cache. App start went to < 500ms.

A8: GENERIC CONTROL TESTING — REFACTOR UNDER PRESSURE

"Tell me about a refactor that unlocked future velocity."

The situation: First version of automated control testing was hardcoded for CC7.1 — the control ID appeared as a literal string in 8 places: the DB query, S3 folder path, prompt template, evidence categorization, report filename, Step Functions input. The founders demoed it. A customer asked: "Can you run this for access control too?"

The problem: Adding a second control would duplicate the entire Lambda. `ControlTester` had `CC7.1` hardcoded in `_initialize_control_requirements()`. DB lookup hardcoded `framework_control_code = 'SOC:CC7.1'`. S3 path was `automated-testing/cc7-1/{tenant}/{timestamp}`.

The two-day refactor:

1. `ControlTester.__init__` now accepts `control_id: str` (the `trustx_id` like `TX-MON04.01`)
2. `_get_database_control_id()` translates: `trustx_id` → `framework_control_code` via lookup table → queries DB by both fields → returns the integer DB ID
3. Control requirements loaded from DB keyed by `trustx_id`, not hardcoded
4. S3 path: `automated-testing/{trustx_id}/{tenant}/{timestamp}`
5. Prompt template: loaded from S3 at `prompts/{trustx_id}/analyze.txt`, parameterized at runtime

After: Adding a new control = data operation (DB row + S3 prompt template). No Lambda code changes. Two more controls were added the following week — I wasn't involved.

What I'd do differently: The `control_id` parameter should have been in the constructor on day one — costs nothing. The hardcoded references were the mistake; the specialized data collectors for CC7.1 were appropriate for a first iteration.

A9: PARALLEL INGESTION CONCURRENCY BUG

> *"Tell me about a bug that only appeared in production."*

Setup: Profile-360 Stage 2 ran 3 Neo4j ingestion Lambdas in parallel. All three wrote to the same per-tenant Neo4j instance simultaneously.

What happened: In dev (1-2 tenants, manual trigger) — no issue. In production (3+ tenants, EventBridge firing all executions simultaneously at 2:00 AM): `ServiceUnavailable: Connection pool size of 50 exceeded`. Each Lambda established its own connection pool at its default size of 50. 3 Lambdas × 50 connections × 3 concurrent tenant executions = 450 potential simultaneous Neo4j connections against instances sized for 50.

The diagnostic: CloudWatch Logs showed the errors starting at exactly 2:00:03 AM — the EventBridge trigger time. The concurrency pattern was invisible in dev and deterministic in production.

Two-part fix:

1. **Short term:** Dispatcher Lambda sits between EventBridge and Step Functions. Reads the tenant list and starts executions with a 30-second stagger between tenants. Slower total runtime, zero simultaneous connections.
2. **Long term:** Set `max_connection_pool_size = 5` in the Neo4j driver config in each Lambda. The default of 50 is for a long-running service — not for a Lambda that handles one request and exits.

The lesson: Concurrency bugs require thinking not just about the concurrency within a single execution, but about the concurrency of executions. Identical EventBridge cron expressions fire simultaneously. Design for it.

A10: PSYCOPG2 PRODUCTION INCIDENT

"Tell me about a production incident you personally resolved."

What happened: AppRunner service started returning 503s. All customer API calls were failing. No warning in dev — the Docker image had passed health checks and deployed successfully.

CloudWatch Logs: `ImportError: /app/lambda_layers/db_utils/python/psycopg2/_psycopg.cpython-311-x86_64-linux-gnu.so: ELF file incompatible with host architecture`. The `.so` path pointed to the Lambda layer directory inside the AppRunner container. That should not exist there.

Root cause: The Lambda layer `db_utils` — built for Lambda's execution environment — was vendored into the AppRunner Docker image via a wildcard `COPY src/ /app/src/`. The Lambda layer lived at `src/lambda_layers/db_utils/python/psycopg2`. Python found two versions of psycopg2 on the import path and picked the Lambda-compiled version — incompatible with AppRunner's architecture. The service didn't fail on `import psycopg2`; it failed on the first actual DB operation after appearing healthy.

Diagnosis path: The `.so` file path in the error was the clue — that directory had no business being there. Checked `sys.path` in a startup log — confirmed both versions were present, Lambda layer version was first.

Fix: Startup cleanup + `.dockerignore`:

```
conflicting_paths = ['/app/lambda_layers/db_utils/python/psycopg2']
for path in conflicting_paths:
    if os.path.exists(path):
        shutil.rmtree(path)
```

The permanent fix: `.dockerignore` excludes `src/lambda_layers/` from the AppRunner build context. The startup cleanup stayed as a safety net.

What I communicated: Founders got a message 20 minutes in: "Service is down, root cause identified, fix deploying, ETA 90 minutes." Second message when restored: "Root cause: X. Fix: Y. Permanent solution: Z." Clear, no jargon.

A11: INSPECTOR V2 INTEGRATION — EXTENDING A LIVE DATA PIPELINE

"Tell me about adding a new data source to an existing system."

Context: Automated control testing collected from 6 sources to answer CC7.1 (System Monitoring). Founders wanted to make the "vulnerability identification" sub-requirement answerable with real vulnerability data, not just inferred from Config rule compliance.

What I added:

- AWS Inspector v2: scans EC2 and ECR images for CVEs, returns findings with severity (Critical/High/Medium/Low) and CVE IDs
- AWS Security Hub: aggregates findings across all AWS security services into a normalized format

The work:

1. IAM role extension in CDK: added `inspector2:ListFindings`, `securityhub:GetFindings`, `securityhub:DescribeHub` to the Lambda execution role policy
2. New `VulnerabilityCollector` class alongside existing `AWSCollector` and `IntegrationCollector`
3. `summarize_data()` in `LLMAnalyzer` aggregates to: count by severity, top 3 critical findings by CVSS score, count of findings older than 30 days (indicates non-responsive vuln management). LLM receives the summary, not the raw finding list.
4. Prompt template updated: added a "Vulnerability Management" section to the CC7.1 template

The general pattern: For any high-volume data source — summarize at the collection layer, not at the prompt layer. The LLM's context window is a scarce resource; don't spend it on raw API output.

A12: GRAPHQL MIGRATION + LIVE SCHEMA CHANGE

"Tell me about changing a data model on a live product."

The original problem: The controls UI made 4–5 sequential REST calls to load a single control detail page — `GET /controls`, `GET /controls/{id}`, `GET /controls/{id}/tests`, etc. N+1 query pattern.

What I built: GraphQL endpoints (Python graphene + Flask) that let the frontend request the exact shape it needed in one call:

```
query {
  control(id: "TX-MON04.01") {
    id trustxId title status
    implementations { id description status }
    tests { id runAt score status
      results { guidelineId verdict evidenceQuote }
    }
    evidences { id type source createdAt }
    comments { id text createdAt author { name } }
  }
}
```

The schema migration it required: `control_evidences` was a flat table attached to controls — no relationship to specific test runs. To make `test.results.evidenceQuote` work in the GraphQL tree, needed to link evidences to test results. Migration: add `test_result_id UUID FK REFERENCES control_test_results(id)` to `control_evidences`, backfill from existing data.

Zero-downtime migration on a live product:

1. Add column as nullable (no constraint yet — doesn't break existing writes)
2. Backfill historical data (matched by timestamp proximity for existing records)
3. Update evidence-generation Lambda to write `test_result_id` on all new records
4. After 2 weeks of confirmed data integrity, add a partial index (not NOT NULL — manually-uploaded evidences genuinely have no test_result parent)

The principle: Add nullable first, backfill, then constrain. Never add a NOT NULL column to a table with existing data in a single migration step.

A13: DOCUMENT LAMBDA — SHIPPING WITH A KNOWN LIMITATION

"Tell me about a time you shipped something that wasn't perfect."

The context: TrustX needed document management for compliance documentation — policies, procedures, evidence artifacts. Documents came in as PDFs and DOCX. The platform needed to display them, allow editing, and save back.

The conversion problem: DOCX round-trip (upload → render in editor → edit → save → DOCX) had 15–20% formatting loss. Tables lost column widths. Lists lost nesting. Specific fonts dropped. Options evaluated: `python-docx` (not lossless), LibreOffice headless (250MB+ Lambda package — at the limit), dedicated AppRunner conversion service (2+ weeks, off-roadmap), Textract (extracts text, not formatting).

What I shipped: Natively-created JSON/text documents work with full fidelity. Uploaded DOCX/PDF: stored as original binary in S3 with a `readonly` flag — displayed in an embedded viewer, editing blocked. Documented explicitly: "Uploaded DOCX/PDF are view-only. Edit compliance documents by creating them in-platform."

Why it was the right call: The compliance team's working documents were already being created natively. Legacy uploads were reference material, not edited in-platform. Shipping a known, explicit limitation is better than shipping a feature that silently corrupts compliance documents.

What I documented in the handoff: Known gap. Recommended path: Mammoth.js (DOCX→HTML with good fidelity) + dedicated AppRunner conversion service. Estimated effort: 2 weeks.

A14: N8N ON ECS — BUILDING WITH AND THEN BEYOND LOW-CODE

"When do you use off-the-shelf tools vs. building your own?"

What n8n was doing: Multi-step compliance report assembly — trigger on schedule, call evidence collection Lambda, receive structured output, pass to OpenAI via HTTP node, format response,

write to PostgreSQL, send Slack notification. Non-engineers could see and modify the workflow in the n8n visual editor.

The Docker platform issue (production bug): n8n image built on Mac (ARM) → pushed to ECR → pulled by ECS (linux/amd64). Error: `image Manifest does not contain descriptor matching platform 'linux/amd64'`. Ran fine locally; ECS couldn't start it. Fix: Docker buildx with explicit `--platform linux/amd64`. Added `setup-docker-buildx.sh` to check for buildx and create a builder before running `docker buildx build --platform linux/amd64`.

Why we migrated to Lambda + Step Functions: n8n was right while workflow patterns were being designed — low-code lets non-engineers iterate quickly. Once patterns stabilized and the team needed tight control over error handling, retry logic, multi-tenant context propagation, and DLQ behavior, Lambda + Step Functions were the better fit. n8n stayed for report-generation workflows (low-frequency, non-critical). Compliance testing workflows moved to code.

The principle: Not every workflow needs to be code. Start with the simplest tool that demonstrates the pattern. Move to code when the pattern stabilizes and the tool's constraints become the bottleneck.

A15: FULL-STACK SHIPPING STORY — THE TEST CONTROL BUTTON

> *Use for "full-stack ownership" or "what's your scope of work?" questions*

The ask: A "Test Control" button in the React dashboard — click it, system runs automated evidence collection and LLM evaluation for a single control, shows the result live.

Four layers in one sprint (5 days):

React UI: Button in controls dashboard. Loading state (spinner + progress counter). Error state (toast). Result panel: verdict badge (color-coded), evidence quote (expandable), confidence score, source references. TypeScript interfaces defined for the API response shape before writing any JSX.

Flask API: `POST /controls/{id}/test` — validates control exists and belongs to the authenticated tenant (JWT → tenant resolution → ownership check), creates job record, publishes to the on-demand SQS queue (separate from nightly batch), returns `{job_id}` with 202. `GET /controls/{id}/test/{job_id}` for polling.

Lambda worker: Evidence collection from 6 sources with asyncio parallel execution, context assembly, Bedrock call with Pydantic schema, evidence-gating post-processing, result written to RDS, job status updated to COMPLETED.

CDK infrastructure: New SQS queue `control-test-queue` (short retention, DLQ, visibility timeout sized for p99 latency), Lambda trigger, IAM role extensions for `Bedrock:InvokeModel` + Secrets Manager access for the 6 source API keys.

No handoffs. I reviewed my own PRs with notes explaining decisions. That's what full-stack ownership means in practice.

PART B — EPIK SOLUTIONS: TECHNICAL

B1: WHAT I BUILT AT EPIK (know this cold)

One-liner: Led AI/ML engineering for a 6-person team. Production LangGraph agentic workflows, Langfuse eval harness, Azure OpenAI + Pinecone RAG, TypeScript React + WebSocket streaming UI.

The context: Epik Solutions built AI-powered workflow automation for enterprise clients. I joined as the AI/ML lead, inheriting a codebase with some OpenAI API calls but no production agentic workflows, no evaluation infrastructure, and no systematic testing approach.

What I shipped:

- LangGraph multi-step agentic workflows with typed state schemas and structured error recovery
- Langfuse eval harness: golden test set, span tracing, regression detection before deploys
- Pinecone + LangChain RAG pipeline for domain-specific knowledge retrieval
- WebSocket streaming UI (React) that surfaces agent reasoning steps live to users
- AWS ECS deployments, Docker, GitHub Actions CI/CD

Team: I led 5 junior/mid engineers. Regular architecture reviews, prompt-engineering pairing sessions, code review on all LLM-adjacent code.

B2: LANGGRAPH AGENTIC WORKFLOW — PHISHING TRIAGE

> *"Walk me through an agentic workflow you built from scratch."*

The problem: A financial services client needed automated phishing email triage — hundreds of reported phishing emails per week. Security analysts were manually triaging each one. The triage involved: extract indicators (URLs, domains, sender info), lookup against threat intelligence feeds, evaluate whether indicators are known-malicious, generate summary recommendation.

The LangGraph architecture:

```
class PhishingTriageState(TypedDict):
    email_content: str
    extracted_indicators: list[Indicator]
    threat_intel_results: dict[str, ThreatResult]
    verdict: Literal["MALICIOUS", "SUSPICIOUS", "BENIGN", "INCONCLUSIVE"]
    reasoning: str
    analyst_note: str | None

# Node graph:
# extract_indicators → lookup_threat_intel → evaluate → generate_summary
# With conditional edges:
# lookup_threat_intel → [multiple parallel lookups] → merge → evaluate
```

State schema discipline: `PhishingTriageState` is a TypedDict with typed fields. Every node declares what it reads and writes. The state schema is the contract between nodes — changing it forces a review of every node that touches it.

Error recovery: If threat intel lookup fails for one indicator (API timeout), the state marks it as `UNKNOWN` instead of failing the entire run. The evaluate node treats `UNKNOWN` as weak evidence — it can still reach MALICIOUS if other indicators are conclusive. The graph never blocks on one partially-failed lookup.

What I care about in agentic systems: Structured state (prevents silent data loss between nodes), typed outputs (Pydantic models at every node output), human-readable intermediate states (every step logs its state snapshot — essential for debugging non-deterministic failures), and fail-partial behavior (one external call timing out shouldn't fail the entire workflow).

B3: LANGFUSE EVAL HARNESS — TESTING LLM OUTPUTS

> *"How did you prevent LLM quality regressions in production?"*

The problem: Before I built the harness, quality changes were detected by users. A prompt tweak would silently degrade verdict accuracy. No systematic test before deploy.

What I built:

Golden test set: 127 labeled examples (email content → expected verdict + expected key indicators). Each example has a human-verified label from the security analyst team. Labels are stored in Langfuse as datasets — version-controlled, not in a spreadsheet.

Span tracing: Every LangGraph node is wrapped in a Langfuse span. The trace shows: node name, input state, output state, LLM call inputs/outputs, latency, token count. A single workflow run produces a complete trace tree.

```
with langfuse.trace(name="phishing_triage", input={"email": email_content}) as
trace:
    with trace.span(name="extract_indicators") as span:
        result = extract_indicators_node(state)
        span.update(output=result)
```

Regression gate in CI/CD: GitHub Actions step after deployment to staging:

1. Run the golden test set against the staging endpoint
2. Compute accuracy, false negative rate (missed malicious = highest risk), latency p95
3. Gate: accuracy must be ≥ 94%, FNR ≤ 2%. Failed gate blocks prod deploy.

The benefit: Prompt changes, model upgrades, and new tool versions are now safe to ship. When the eval gate catches a regression, the Langfuse trace pinpoints exactly which node changed behavior.

What I tell teams: Build the golden set before you optimize. You can't measure a regression from a baseline you don't have. The first 25 examples are worth a full sprint.

B4: PINECONE + LANGCHAIN RAG — KNOWLEDGE RETRIEVAL

> *"Tell me about a RAG system you built."*

The problem: The phishing triage system needed to compare indicators against an internal threat intelligence knowledge base — thousands of known-malicious domains, IP ranges, pattern descriptions, historical incident summaries. Full knowledge base didn't fit in context. New entries added daily by analysts.

What I built:

- **Ingestion pipeline:** New threat intel entries chunked (512 tokens, 50-token overlap), OpenAI `text-embedding-3-small` embeddings, stored in Pinecone with metadata: `{source_id, entry_type: "domain"|"ip"|"pattern"|"incident", severity, date_added}`
- **Retrieval:** `PineconeVectorStore` + `LangChainRetriever` with a metadata filter: `{entry_type: "domain"}` for domain lookups, `{entry_type: "pattern"}` for signature matching. Filter reduces retrieval to the relevant slice before semantic search.
- **Multi-query retrieval:** For ambiguous indicators, generate 3 query variations (paraphrase via LLM) and merge results. Deduplicate by `source_id`. Handles cases where "typosquat domain" and "lookalike URL" are different analysts' terminology for the same thing.

Pinecone vs. pgvector decision: pgvector was already in the stack. The reason to use Pinecone here: the analyst update pipeline was frequent (daily writes) and the team managing it was separate from the backend engineers. Pinecone's Python SDK let the analyst tooling team write directly without PostgreSQL migration access. When daily writes slow down and the team is comfortable with the DB, it's a straightforward migration — same query patterns, different client.

Production tuning: Embedding model choice matters more than vector DB choice. `text-embedding-3-small` → `text-embedding-3-large` gave +4% retrieval accuracy on our eval set at 6x cost. We kept small for production, large for high-confidence re-queries when first pass score < 0.75.

B5: TYPESCRIPT IN PRACTICE — WHAT IT ACTUALLY BUYS YOU

> *Use for "TypeScript experience" questions or EM1 stack alignment*

Where I used it at Epik: React frontend (streaming UI, LangGraph state visualization), Node.js API gateway (request validation with Zod, route type inference with tRPC), some shared type definitions consumed by both.

The patterns I actually use:

Discriminated unions for workflow state:

```
type AgentStatus =
```

```
  | { phase: "extracting"; indicators_found: number }
  | { phase: "looking_up"; pending: string[]; completed: string[] }
  | { phase: "evaluating"; confidence: number }
  | { phase: "complete"; verdict: "MALICIOUS" | "SUSPICIOUS" | "BENIGN";
summary: string }
  | { phase: "error"; message: string; recoverable: boolean }

// Exhaustive switch with no fallthrough:
function renderStatus(status: AgentStatus) {
  switch (status.phase) {
    case "extracting": return <ExtractingView count={status.indicators_found} />
    case "looking_up": return <LookupView pending={status.pending} />
    // ... TypeScript error if any phase is unhandled
  }
}
```

Zod at API boundaries: Runtime validation at the entry point: `AgentResponseSchema.parse(apiResponse)`. Fails fast with a clear error if the Python backend changes its response shape — doesn't silently carry malformed data through the UI.

Why it matters for EM1: TypeScript is the EM1 stack. I'm not learning it on the job. The shared type discipline — Python Pydantic schema ↔ TypeScript Zod schema for the same API contract — is a pattern I'd bring to the EM1 codebase.

PART C — BEHAVIORAL STORIES

All behavioral answers consolidated, organized by question type

> **Practice these out loud.** Behavioral questions come fast. Know your opening sentence for each.

C1: OWNERSHIP / END-TO-END

"Tell me about a time you took full ownership of a project."

Situation: Samay hired me as their first and only engineer. No existing codebase. The founder had a compliance SaaS concept: automate SOC2 evidence collection, run LLM evaluation, produce auditor-ready reports. I joined with a GitHub org, an empty AWS account, and a requirements doc.

Task: Design the architecture, build it, ship it to paying customers — without a senior peer to review my decisions.

Action: I started with the customer problem, not the tech. Spent 2 weeks reading SOC2 audit requirements, talking to the founders about what auditors actually ask for, and mapping data sources to control requirements. Built a lightweight prototype first — hardcoded CC7.1, one Lambda, manual trigger, rough React dashboard. Showed it to the founders, got signal, then built the production version. Structured the CDK stacks so each concern was isolated: network, database, compute, AI pipeline, tenancy. Wrote decision logs for every major architectural choice — not for anyone else, for myself, so I could revisit them when requirements changed.

Result: 15+ CDK stacks, 6 evidence sources, LLM scoring with evidence-gating, React dashboard, 3 paying B2B customers, comprehensive handoff document. Built in 9 months solo.

What I learned: Full ownership doesn't mean doing everything in isolation — it means ensuring no decision gets dropped. I was the only one accountable for the database schema, the prompt design, the tenant isolation model, the IAM permissions. That focus creates quality. It also creates blindness. I wish I'd had code review on architecture decisions — not implementation, just architecture.

C2: WORKING WITH DOMAIN EXPERTS

"Tell me about working with a non-technical stakeholder or domain expert."

Situation: At Samay, one of TrustX's early customers brought in their compliance auditor as an advisory reviewer — someone who had conducted hundreds of SOC2 audits. The founders asked me to meet with him to validate the LLM evaluation outputs.

Task: Get honest feedback from an expert who had no reason to soften criticism — and translate it into actionable product changes.

Action: I prepared by pulling 20 real evaluation outputs — a mix of PASSes, FAILs, and what I considered edge cases. I showed them without framing, just "here's what the system said." His reaction to the first PASS: "This is correct, but I couldn't defend this in front of a regulator — the evidence quote is too vague. I need a clause reference, not a paraphrase." That single comment rewrote my mental model of what "evidence" meant.

I took notes, not just on his verdicts but on his reasoning. He used phrases like "defensible in writing," "maps to a named sub-requirement," "shows pattern of behavior vs. one-time." I rewrote the prompt templates around those criteria. Added `source_reference` to the Pydantic schema — not just the quote, but the specific document and section. Added a `sub_requirement_id` field that forced the LLM to cite which sub-control it was assessing.

Result: Second review session, his pass rate on the LLM outputs went from ~60% to ~85%. He said: "I can now show a client this output and tell them it's defensible." That was the real acceptance criteria.

What I took from it: Technical correctness and expert usability are different bars. Build for the expert workflow, not the technically correct workflow. Domain experts tell you the implicit requirements that no spec document captures.

C3: PRODUCTION INCIDENT — SAMAY (PSYCOPG2 503)

"Tell me about a production incident you led."

> *Technical root cause is in A10 — this answer focuses on response and communication.*

Situation: AppRunner service — the customer-facing API — started returning 503s on a Tuesday morning. All 3 customers affected simultaneously.

Task: Diagnose, communicate, fix, prevent recurrence. No runbook. No on-call rotation. Just me.

Action (first 20 minutes): Checked CloudWatch Logs immediately. Error was clear: `ImportError` on a `.so` file from the Lambda layer directory. Didn't panic — the error was specific. Root cause was identifiable. Sent a message to the founders: "Service is down. Root cause identified: wrong psycopg2 version being loaded in the AppRunner container due to a path conflict from the Lambda layer. Fix is in progress. ETA: 90 minutes."

The fix (90 minutes): Added startup cleanup code to purge the conflicting path. Added `.dockerignore` to prevent it permanently. Rebuilt Docker image, pushed to ECR, triggered AppRunner redeployment.

Communication (after restore): Founders got a second message: "Service restored at 10:47 AM. Total downtime: 94 minutes. Root cause: Lambda layer directory was incorrectly included in the Docker build context for AppRunner. Fix: startup path cleanup + .dockerignore exclusion. This class of path conflict cannot recur."

Result: Both founders appreciated the communication cadence — they knew what was happening the entire time and could answer customer questions if needed. The incident became the forcing function for properly separating Lambda and AppRunner deployment contexts in the CDK stack.

What I'd do differently: Add a startup integration test to the AppRunner container health check that verifies `import psycopg2; psycopg2.connect(...)` succeeds before the container is considered healthy. Would have caught this before it hit production.

C4: PRODUCTION INCIDENT — EPIK (SILENT DATA DROP)

"Tell me about a time data quality was silently degraded."

Situation: At Epik, we had a data processing pipeline using Spark on EMR — processing client log files and writing structured output to S3 for the LLM context retrieval. After a routine Spark version upgrade, a stakeholder noticed "the context seems shallower" — the LLM responses felt less grounded in recent data.

Task: Investigate whether recent data was actually missing from the context, not just a perception issue.

Action: Pulled the S3 output partition listing. Cross-referenced against the source log file count. 15–20% of records were missing from the most recent 3 weeks. Spark had silently dropped records with null values in a field that was previously optional but had become required in the new version's type inference — no exception, no error log, just schema coercion and row-level drops.

The fix: Explicit schema definition for the input files (no type inference). Added a record count check: after each Spark job, compare input record count (from S3 source) to output record count. Delta > 0.5% fails the job and alerts. Reprocessed the 3-week backfill.

What I communicated: Wrote a clear incident summary for the team: "15–20% record drop in weeks X–Y. Root cause: Spark schema inference behavior change in [version]. Fix: explicit schema + count gate. Backfill complete. Future drops will alert."

The lesson: Silence from a data pipeline is not confirmation of correctness. Build a count invariant

at every stage. An alert on "zero records" or "anomalous drop" is non-optional for any data pipeline feeding an LLM context.

C5: CHANGING REQUIREMENTS

"Tell me about a time requirements changed mid-project."

Situation: At Samay, the initial TrustX scope was SOC2 only — CC6 and CC7 controls. Two months into development, the founders came back after customer conversations: a prospect wanted NIST 800-53 and ISO 27001 support. The evidence collection patterns were SOC2-specific. The control taxonomy was hardcoded to SOC2 notation.

Task: Pivot the framework architecture without rebuilding from scratch — and without breaking the SOC2 work already done.

Action: First step was an audit: what was genuinely SOC2-specific vs. what was accidentally SOC2-specific due to hardcoding? The evidence collection patterns (GitHub commit analysis, Jira ticket analysis, AWS Config rule evaluation) were not framework-specific — they're answering "does this control category have evidence?" The framework specificity was only in the control taxonomy, control IDs, and prompt templates. All three could be moved to data.

Introduced a `frameworks` table and a `controls` table keyed by `framework_code` (SOC2, NIST800-53, ISO27001) and `control_id`. Prompt templates stored in S3 at `prompts/{framework}/{control_id}/analyze.txt`. The Lambda received `{framework_code, control_id}` and loaded everything dynamically.

Result: NIST 800-53 support was added in 10 days. The Lambda code didn't change — only new DB rows and S3 prompt templates. ISO 27001 was added the following week by the founders themselves using the same pattern.

What I'd do differently: The multi-framework architecture should have been the starting design. SOC2-only was an explicit MVP decision, and that was right. But the hardcoding of SOC2 notation in the Lambda (not in data) was accidental — could have been data-driven from day one at no extra cost.

C6: HARD ARCHITECTURAL DECISION

"Tell me about a hard technical decision you had to make."

> *See A4 for full technical depth — this is the STAR framing.*

Situation: TrustX needed to answer identity traversal questions: "who can access this S3 bucket and through what permission chain?" The chain involved 4–6 hops: user → role → IAM policy → resource action → S3 bucket ARN. PostgreSQL with a recursive CTE was my first implementation.

Task: Decide whether to add Neo4j — a second database, a new skill to learn, a new CDK construct to build and maintain — based on query performance that was "slow but correct."

Action: The PostgreSQL CTE took 800ms for medium-complexity identity chains — 30 seconds

for the multi-user, multi-resource questions the auditors actually asked. I built the Neo4j version on a branch, loaded the same test data, ran the same queries. 20ms. 40x difference.

I documented the decision before implementing: "Adding Neo4j. Accepted cost: Cypher ramp-up (estimated 1 week), CDK construct for Neo4j on EC2, per-tenant instance model, data sync between RDS and Neo4j. Benefit: 40x query time, sub-100ms identity traversal that's interactive for the auditor. Alternatives rejected: PostgreSQL CTE optimization (already at index-limit), graph extension for Postgres (RDS doesn't support it)."

Brought it to the founders as a decision, not as a proposal. Here's the tradeoff. Here's why I think it's worth it. They said yes.

Result: Interactive identity traversal in the dashboard. Auditors could ask "show me everyone with write access to the production S3 bucket" and get a result in real time. The Neo4j construct became a standard part of the tenant onboarding stack.

What I'd do now differently: The same decision rule — add complexity only when the simpler version demonstrably fails at scale. I'd still make the same call.

C7: TECHNICAL PUSHBACK

"Tell me about a time you pushed back on a technical direction."

Situation: At Epik, a client's IT team proposed using Workato — a no-code integration platform — to orchestrate the multi-step AI workflow we were building. Their reasoning: their IT team could maintain it without needing engineers.

Task: Evaluate whether Workato was the right tool, and if not, push back with evidence rather than instinct.

Action: Spent a weekend building a proof of concept of our phishing triage workflow in Workato. Not to prove it wouldn't work — to understand what "it doesn't work here" actually meant in practice. Specific blockers:

1. Workato's recipe steps are sequential. Our LangGraph workflow had parallel lookups with a merge step. Not possible in Workato without expensive workarounds.
2. Prompt templates in Workato are string fields in a recipe. No version control, no PR review, no testing. A changed prompt deploys immediately to production.
3. No way to implement the Langfuse span tracing — we'd lose observability on LLM call inputs/outputs.

Brought the findings to the client's IT lead and my manager: "Here's what works in Workato. Here's what doesn't. Here's what it costs us in observability and reliability to use it for this use case." Proposed a compromise: Workato for upstream data collection (where their team had genuine ownership) + LangGraph for the AI evaluation step (where we had the expertise and needed the control).

Result: Client accepted the split architecture. Their IT team maintained the Workato recipes for data ingestion. We maintained the LangGraph workflow. No handoff confusion — clean ownership

boundary.

What I learned: Pushback works when it comes with evidence and a compromise. "This is the wrong tool" without a proof of concept is just an opinion.

C8: TECHNICAL DEBT DECISION

"Tell me about a time you had to decide whether to pay down technical debt."

Situation: At Samay, the original TrustX deployment was done with shell scripts before I migrated everything to CDK. The shell scripts still existed — commented out, but present in the repo. More critically, the Step Functions state machine and several Lambdas had been initially deployed via the AWS console and were still not fully under IaC. A growing divergence between what CDK described and what actually existed in the AWS account.

Task: Decide when and how much to invest in migration — while still shipping product features.

Action: I audited the divergence: listed every resource that was console-managed vs. CDK-managed. Categorized by risk: "console-only, would fail to recreate if destroyed" (critical) vs. "CDK has the resource but console has manual modifications" (medium) vs. "shell script creates it but CDK also manages it" (low).

Migrated the critical resources first — 3 sessions of a few hours each. The Step Functions state machine was the hardest: the JSON definition had to be reverse-engineered from the console's visual editor, then built into the CDK `WorkflowLoader` pattern. Flagged the medium-risk resources with a `#TODO: CDK-migrate` comment and a GitHub issue, addressed them in the next sprint. Left the shell scripts in place as documentation (noted in README: "historical deployment scripts — superseded by CDK, kept for reference").

Result: After 2 sprints, every production resource had a CDK definition. The account could be recreated from CDK if needed. That became the definition of "IaC complete" that I wrote into the handoff document.

The principle: Technical debt that creates operational risk is always worth paying. Debt that creates future velocity loss is worth scheduling. Debt that's just aesthetic — fix it when you're near it.

C9: SHIPPING WITH KNOWN CONSTRAINTS

"Tell me about a time you shipped something with a known limitation."

> *See A13 for full technical context — this is the STAR framing.*

Situation: TrustX needed document management for compliance documentation. Customers needed to upload legacy Word docs, display them, edit them in-platform, and save back to DOCX. I hit a hard technical constraint: DOCX round-trip had 15–20% formatting loss with every option I evaluated.

Task: Decide whether to block the feature, ship with known loss, or ship with the limitation made

explicit.

Action: Tested 4 options: `python-docx` (not lossless), LibreOffice headless (250MB+ Lambda — at the limit), dedicated AppRunner service (2+ weeks, off-roadmap), Textract (text only, no formatting). None gave acceptable DOCX fidelity.

Proposed to the founders: ship document management for natively-created documents (full fidelity) + view-only mode for uploaded DOCX/PDF with an explicit UI label: "Uploaded files are view-only. To edit compliance documents, create them in-platform." Documented the gap clearly in the product — not in a hidden tooltip, in the section header.

Their response: "This is fine for now. Our current customers are mostly creating documents natively." Shipped it.

What I wrote in the handoff: This was a known gap. Recommended path: Mammoth.js for DOCX → HTML (best fidelity I found), dedicated AppRunner conversion service. Estimated 2 weeks. Flagged as "worth doing when document workflows become a customer retention issue."

What I learned: Shipping with an explicit limitation is not the same as shipping broken software. An honest "this doesn't do X yet" is better than a feature that silently corrupts compliance documents. Make the constraint visible. Give the team a path forward in writing.

C10: MENTORING JUNIOR ENGINEERS

"Tell me about a time you helped a junior engineer grow."

Situation: I was a GTA (Graduate Teaching Assistant) for a distributed systems course at OSU, and also did informal mentoring at Zemoso Technologies with junior engineers on my team.

The OSU context: Weekly office hours with students who were stuck on distributed consensus implementations — Raft, leader election, log replication. The gap wasn't usually the algorithm — it was test strategy. Students wrote happy-path tests, then were surprised when their implementation failed on network partition tests.

What I actually did: Didn't give answers. Gave probes: "Your test simulates a leader failing. What happens to a client request that was in-flight at the moment of failure? Does your test capture that?" Watched them realize the gap in their mental model, not my mental model. Most of the time, when they could name the failure scenario precisely, they could implement the fix themselves.

The Zemoso context: Paired with a junior engineer on a data pipeline — they kept writing queries that worked on the test database (small, homogeneous data) and failed in production (large, heterogeneous, nulls everywhere). Instead of telling them "add null handling," I asked them to describe what the query assumed about the data. Once they listed the assumptions, they found the null case themselves.

The pattern I use: Ask what they assume to be true. Most bugs come from an assumption that isn't — and pointing out the assumption is more durable than pointing out the fix.

C11: DESIGNING FOR HANDOFF

"Tell me about a time you prepared your work to be maintained by others."

Situation: At Samay, I knew from month 6 that my role was time-bounded — the company was growing and would eventually hire a backend team. I started treating "handoff-readiness" as a design criterion alongside correctness and performance.

Task: Ensure that what I built could be taken over by engineers who hadn't been in every architecture conversation.

Action: The handoff document (`Neel_TrustX_ATAP_Comprehensive_Handoff_Document.md`) wasn't written at the end — it was maintained throughout. Every major architectural decision had a section: what was built, why that approach was chosen, what alternatives were rejected, what known gaps remained, what the recommended next step was.

Specific choices made for handoff-readiness:

- Every CDK stack had a docstring: "This stack deploys [X]. Dependencies: [Y]. Known limitations: [Z]."
- Lambda functions had comments for non-obvious sections (why the path cleanup ran at startup, why lazy loading was implemented for tenant mappings)
- The `.dockerignore` list had comments: "Excluded: src/lambda_layers/ — Lambda packages are ARM-compiled and incompatible with ECS/AppRunner AMD64."
- The Step Functions state machine JSON had inline comments (not valid JSON but preserved in the source `*.json.template` file alongside the compiled version)

Result: The handoff document was the artifact that Samay showed to prospective engineers in their hiring process. It became a selling point: "This is what you're inheriting — it's documented."

What I believe: A handoff document is not bureaucracy. It's engineering discipline. Future-you-or-someone-else is a user of your system. Design for them.

C12: DOMAIN ONBOARDING — LEARNING A NEW FIELD

"How do you get up to speed in a domain you don't know?"

Situation: At Epik, a new client engagement was in financial fraud detection — a domain I had no background in. The client's team used specialized terminology (typology, layering, placement, KYC, SAR). They assumed domain fluency I didn't have.

Task: Get operational fast without pretending to know what I didn't — and build enough fluency to ask real questions, not generic ones.

Action: My first week was observation, not contribution. Sat in on client calls without speaking. Took notes on terminology without immediately looking it up — heard how the words were used in context before reading the definitions. By the end of the week, I had a personal glossary of 40+ terms with their usage contexts.

When I had a question, I asked the client's SMEs — not Google, not the team. Specific questions: "When you say a transaction is 'structuring', is that a behavioral pattern, a regulatory classification, or both?" Those questions signaled that I was engaging with the actual domain, not just engineering a system on top of it.

Result: By week 3, I was able to design the data schema for the transaction flagging system using the client's terminology — not mine. The client commented: "You understood what we're actually doing faster than most vendors we've worked with."

The rule I follow: Watch the work first. Read the glossary second. Ask questions third. Implement last. Reversing this order is how engineers build systems that are technically correct but domain-wrong.

C13: MOTIVATION — WHY EM1

"Why EM1? Why this role?"

Three honest reasons:

1. The domain: Compliance and government resilience are not the same problem. EM1 is building infrastructure for how governments plan for disasters — that's more consequential than most SaaS. FEMA 111-guideline evaluation at scale means analysts get hours back per plan review. That's real. I want to build things where the outcome matters beyond the business metrics.

2. The technical fit is exact, not approximate: The TrustX evaluation pipeline is structurally identical to EM1's Smart References problem — evidence grounding, hallucination prevention, multi-document context assembly, human-in-the-loop review. I built the Samay version; I understand what breaks in production. That's not a "transferable skill" — it's the actual thing.

3. The team size and stage: 4 engineers, real customers, a problem that's just now becoming technically tractable with modern LLMs. This is the stage where the architectural decisions made in the next 12 months will define the system for 5 years. I want to be in the room for those decisions. I've been the sole engineer at that stage — I know how to move fast and build things that last.

What I'm not pretending: I don't know Terraform deeply yet. I'll learn it. I don't know the EM1 codebase. I will. The things I bring — evidence-gated AI evaluation, multi-tenant architecture, full-stack shipping without handoffs, building for domain experts — those are real and ready now.